The Silent Pavilion

4분 라이브 데모 발표 대본 · 슬라이드 없이 Unity와 실제 코드로 발표

> **발표 목표 3분 50초~4분.** 대본은 인쇄물이나 별도 기기에서 보고, 노트북에는 Unity와 VS Code만 띄웁니다. 58초 영상은 비상 상황에서만 사용합니다.

발표 전에 띄워 둘 화면

Unity

- **TheSilentPavilion.unity**를 엽니다.
- Game 탭 16:9, Play 후 외부 시작점 대기.
- 클릭: 마우스 고정 / Esc: 해제.
- Console 오류를 지웁니다.

VS Code 탭 순서

- **MidMeshBuilder.cs** 24~65행
- **MidCreatureBreathing.cs** 30~49행
- **MidMatrixMath.cs** 18~31행
- 글자 18~20pt, Explorer·터미널 닫기.

화면 전환 동선

시간	띄울 화면	핵심 내용
0:00~0:25	Unity Game · 외부	작품 콘셉트
0:25~0:55	Unity Game · 정자로 이동	장면 구성과 구현 범위
0:55~1:45	MidMeshBuilder.cs	반복문 Mesh 생성
1:45~2:45	Breathing.cs → MatrixMath.cs	직접 행렬 변환
2:45~3:30	Unity Game · 근접	결과와 추가 요소 2개
3:30~4:00	Unity Game · 정지 화면	구현 범위와 마무리

비상 화면

Assets/Mid/Submission/The_Silent_Pavilion_Midterm.mp4를 미리 열어 둡니다. Unity가 재컴파일되면 25초 부근부터 재생하면서 2:45 이후 대본을 이어갑니다.

장면을 보여 주고, Mesh 코드로 전환

0:00–0:25 작품 소개

[띄울 화면] Unity Game. 정자 외부 전경에서 2초 정도 움직이지 않습니다.

안녕하세요. 제 중간 프로젝트는 **The Silent Pavilion, 침묵의 정자**입니다. 밤의 숲을 지나 정자에 다가가면, 원래부터 그곳에 앉아 있던 것 같은 나무형 크리처가 서서히 드러납니다. 큰 점프 스케어보다, 가만히 있던 존재가 아주 조금씩 움직일 때 생기는 불안함을 표현했습니다.

[조작] 마지막 문장에서 W를 눌러 천천히 정자로 이동합니다.

0:25–0:55 장면 구성

[띄울 화면] 정자로 걸으며 숲과 랜턴이 함께 보이게 합니다.

장면에는 정자와 기존 벤치, 직접 만든 크리처, 뒤틀린 숲, 네 개의 불빛과 랜턴이 있습니다. 정자와 벤치는 기존 에셋을 재사용했고, 크리처는 별도 의자가 아니라 정자의 원래 벤치에 앉혔습니다. 과제에서 설명할 직접 구현 부분은 크리처와 숲의 메시, 크리처의 호흡 변환, 그리고 불빛의 움직임입니다.

[조작] 크리처가 보이기 시작하는 위치에서 멈춤 → Esc → Alt+Tab으로 VS Code.

0:55–1:45 반복문으로 생성한 Mesh

[띄울 화면] MidMeshBuilder.cs 24~65행. 34행 → 51행 → 61~65행 순서로 가리킵니다.

필수 첫 번째는 C#으로 직접 생성한 메시입니다. 이 함수는 제어점을 통과하는 Catmull–Rom 곡선을 가지의 중심선으로 사용합니다. 34행의 바깥 반복문은 중심선을 따라 이동하고, 51행의 안쪽 반복문은 각 지점의 단면 둘레를 돕니다. 여기서 중심점에 단면 방향과 반지름을 더해 실제 정점 위치를 만듭니다.

단면마다 반지름과 회전, 불규칙한 능선을 바꿔서 원기둥처럼 보이지 않고 가늘고 뒤틀린 가지가 되도록 했습니다. 그리고 61행부터 이웃한 두 단면의 정점 네 개를 삼각형 두 개로 연결합니다. 이 생성기를 크리처의 몸과 별도의 나무 형태에 사용했습니다.

[다음] MidCreatureBreathing.cs 탭으로 전환합니다.

행렬 코드를 보여 주고 결과로 복귀

1:45–2:45 직접 구현한 행렬 변환

[화면 A] MidCreatureBreathing.cs 30~49행. 30행의 순서와 44·48행의 정점 반복문.

필수 두 번째는 직접 구현한 행렬 변환입니다. 크리처의 골반 근처를 피벗으로 잡고, 먼저 피벗을 원점으로 옮긴 뒤 크기 변화와 두 축 회전을 적용하고 다시 원래 위치로 되돌립니다. 크기와 각도는 사인 함수로 계산해서 상체가 천천히 호흡하고 좌우로 흔들립니다.

매 프레임 이전 결과가 아니라 원본 정점에 현재 행렬을 적용하기 때문에 변형이 계속 누적되지 않습니다. 눈에도 같은 행렬을 적용해서 얼굴과 따로 움직이지 않게 했습니다.

[2:15 조작] MidMatrixMath.cs 탭으로 전환합니다.

[화면 B] MidMatrixMath.cs 18~31행. 22~23행의 3중 반복문과 26~31행.

이 부분이 Unity의 TRS나 MultiplyPoint를 대신하는 직접 계산입니다. 4×4 행렬 곱은 행과 열의 내적을 3중 반복문으로 계산하고, 동차좌표의 이동 성분까지 포함해 각 정점을 변환합니다.

[조작] Alt+Tab으로 Unity 복귀 → Game 화면 클릭.

2:45–3:30 게임 화면에서 결과 확인

[띄울 화면] 크리처 가까이에서 4~5초 정지한 뒤 바깥 숲과 불빛 쪽으로 천천히 시선을 돌립니다.

게임 화면으로 돌아오면 방금 설명한 정점 변환 때문에 상체와 머리가 아주 작게 움직이는 것을 볼 수 있습니다. 추가 선택 요소는 두 가지입니다. 첫째, 크리처와 다른 분기 구조를 가진 뒤틀린 나무 숲을 별도의 기하 모델링 대상으로 만들었습니다. 둘째, 떠다니는 불빛은 크리처 변형과 별개로 사인과 코사인을 사용한 타원 궤도와 상하 운동을 합니다.

플레이어가 가까워질수록 크리처가 보이고 짧은 화면 잡음이 발생합니다. 걷기 효과는 실제 이동 거리에 따라 시점 흔들림과 발소리 주기를 계산합니다.

[조작] 크리처와 랜턴이 함께 보이는 구도로 돌아와 멈춥니다.

3:30–4:00 마무리

정리하면, 이번 프로젝트에서는 반복문으로 만든 유기적인 메시와 직접 계산한 행렬 변환을 장면의 핵심으로 사용했습니다. 여기에 별도 나무 모델링과 독립된 불빛 운동을 추가했습니다. 수피의 색과 홈은 직접 작성했지만 빛 반응은 현재 URP PBR을 사용하며, 직접 조명식과 깊이 기반 안개는 기말 확장 계획입니다. 이상으로 발표를 마치겠습니다. 감사합니다.

발표 직전 확인과 예상 질문

직전 체크리스트

- ☐ Mid 브랜치의 완성 장면인가?
- ☐ Play 후 외부 시작점인가?
- ☐ WASD·마우스·Esc가 되는가?
- ☐ 코드 세 탭의 행 위치가 맞는가?
- ☐ 글자가 옆자리에서도 읽히는가?
- ☐ 알림을 끄고 Console을 비웠는가?
- ☐ 비상용 MP4가 열려 있는가?
- ☐ 화면 전환 포함 4분 리허설을 했는가?

데모가 꼬였을 때

- 마우스가 갇히면 **Esc**.
- 시간이 밀리면 MatrixMath 화면을 생략하고 직접 행·열 내적이라고 말합니다.
- Unity 재컴파일 시 MP4 25초 부근부터 재생합니다.
- 20초 이상 부족하면 보행 설명을 생략하고 정리합니다.

정상 진행에서는 58초 영상을 재생하지 않습니다. 전체 영상까지 재생하면 발표 시간이 넘습니다.

예상 질문

Q. 크리처 전체가 직접 만든 메시인가요?
A. 몸과 가지, 머리와 관절을 코드로 생성했습니다. 정자와 기존 벤치 및 일부 텍스처는 재사용 에셋이며 직접 구현 항목으로 계산하지 않았습니다.

Q. 행렬 순서가 왜 중요한가요?
A. 열벡터 기준 오른쪽부터 적용되며, 피벗을 원점으로 옮긴 상태에서 크기와 회전을 적용해야 골반 근처를 중심으로 움직입니다. 순서를 바꾸면 회전 중심과 최종 위치가 달라집니다.

Q. 조명 셰이더도 직접 구현했나요?
A. 현재 수피의 색과 홈, 화면 잡음은 직접 작성했지만 광원에 대한 PBR 계산은 URP 함수를 사용합니다. 직접 조명식은 기말 계획입니다.

Q. 추가 선택 요소 두 개는 무엇인가요?
A. 별도 나무 숲의 기하 모델링과 독립된 불빛의 궤도 변환입니다. 필수 크리처 메시와 호흡 변환과는 대상과 결과가 다릅니다.

> **발표 팁:** 코드를 모두 읽지 말고, 화면에서 바깥 반복문 → 안쪽 반복문 → 삼각형 연결, 행렬 순서 → 정점 적용 순서만 손으로 가리킵니다.